
結果の取得と出力 その1

SensitiveDetector

Geant4 11.3.0 準拠

Geant4講習会資料: HEP/NP/Space 2025



大学共同利用機関法人

高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

本講の目標

- Geant4のスコアリングの手法を実例を通じて理解する
 1. G4Stepなどから幾何学情報や物理情報の取得
 2. Sensitive Detector (有感検出器)とその使い方

参考にする他のコース

「構造体の形状定義」「ユーザクラスの記述方法」

Standard Template Libraryの教科書も

Geant4のアプリ作成には3つの必須クラスの実装が必要とされる。スコアリングではさらに、Geant4カーネルに介入するために必要なだけユーザ・クラスを実装しなければならない。この意味で、スコアリング・プログラミングは今までの知識を集大成して初めて可能になるといえる。

本講義はテーマが多いのでL09, L10と2部に分けてあるが内容は連続しており、2つで完結する。対応して二つの演習H09, H10があるがそれも連続している。

目次

1. スコアの取得
 1. G4Step
 2. G4Track
 3. トラッカーとカロリメータの例
2. ユーザアクション
3. Sensitive Detector 有感検出器
 - 有感検出器とは
 - 有感検出器の実装とジオメトリの関連
4. 既製の多機能検出器と原子スコアラ

スコアの取得 ; G4Step

測定器のミッションは多様である

- 位置の測定
 - ✓ 電子のドリフト移動とガス増幅とを使う
Proportional chamber 比例係数箱 PC, Multi-Wire PC, Cylindrical MWPC, Drift Chamber, Time Projection Chamber etc.
- 時間の測定
 - ✓ 主に高速のシンチレータを使う
- エネルギーの測定
 - ✓ 電磁シャワー
全吸収型、サンプリング型
 - ✓ ハドロンシャワー
- 運動量の測定
 - ✓ 磁場中に位置測定器を置く
- 粒子の同定
 - ✓ Time_Of_Flight, Cherenkov Counter, Transition Radiation, dE/dX など

スコアリングとその基本

■ スコアリング

- 各種に検出器が生み出す物理量を得て、外部ファイルへ出力する課題。
 - ・ どの検出器？ ジオメトリ
 - ・ どのような物理量？ ステップ

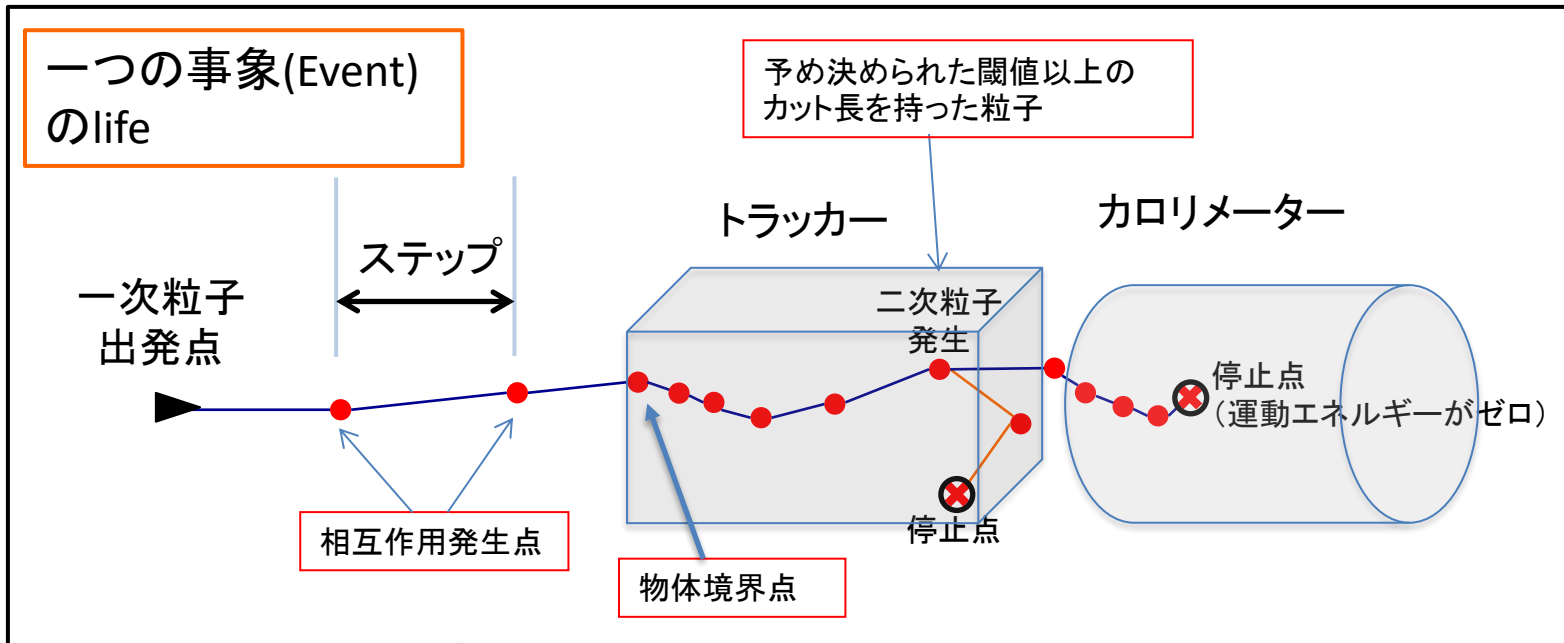
■ Geant4のスコアリングの基本

- すべてのスコアはステップから得られる
 - ・ ステップの時空座標
 - ・ トラックの運動量とエネルギー
 - ・ ステップでのエネルギーデポジットとステップの長さ
 - ・ ジオメトリ情報
 - ・ それ等の組み合わせ
- 相互作用で発生した二次粒子のステップを追いかけるかどうかの判断などシミュレーションの制御に必要な機能（本稿では省略）

■ 本稿ではGeant4のスコアリングの基本について詳しく解説する

復習： G4Stepはシミュレーションの最小単位

- シミュレーションは粒子進路に沿ったステップ(step)単位で進行する
 - ✓ ステップ単位で粒子と物質の相互作用(物理: physics)が考慮される
 - ✓ 粒子が構造物の境界面を通過すれば、通過点で一つのステップは終了
 - ✓ 粒子の1ステップ長だけの時空間移動 (transportation) も相互作用として扱う
 - ✓ 粒子は運動エネルギーがゼロになれば停止し、ステップの進行は終了
 - ✓ 閾値(カット長)以下の二次粒子はステップを発生させない **これ大事**
- **G4step**オブジェクトはstepの長さ、エネルギー損失などstep全体に関する情報を保持する



G4Step.hhを読む

一度はソースに目を通して



Geant4 Cross Reference

[Cross-Referencing](#) [Geant4](#)
[Geant4/](#)

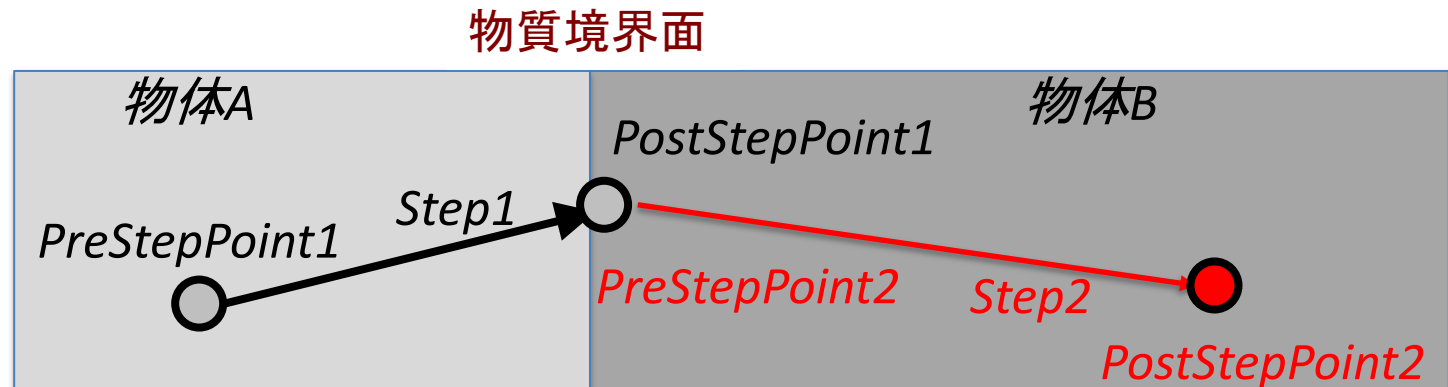
Version: [[ReleaseNotes](#)] [[1.0](#)] [[1.1](#)] [[2.0](#)] [[3.0](#)] [[3.1](#)] [[3.2](#)] [[4.0](#)] [[4.0.p1](#)] [[4.0.p2](#)] [[4.1](#)] [[4.3.p1](#)] [[4.3.p2](#)] [[9.0](#)] [[9.0.p1](#)] [[9.0.p2](#)] [[9.1](#)] [[9.1.p1](#)] [[9.1.p2](#)] [[9.1.p3](#)] [[9.2](#)] [[9.2.p1](#)] [[10.0.p4](#)] [[10.1](#)] [[10.1.p1](#)] [[10.1.p2](#)] [[10.1.p3](#)] [[10.2](#)] [[10.2.p1](#)] [[10.2.p2](#)] [[10.2.p3](#)] [[10.3](#)] [[11.0.p3](#)] [[11.0.p4](#)] [[11.1](#)] [[11.1.1](#)] [[11.1.2](#)] [[11.1.3](#)] [[11.2](#)] [[11.2.1](#)] [[11.2.2](#)] [[11.3.0](#)]

Name	
	analysis/
	cmake/
	config/
	digits_hits/
	error_propagation/
	event/
	examples/
	externals/

- Pointers to **PreStep** and **PostStepPoint**
- **Geometrical step length** (step length before the correction of multiple scattering)
- **True step length** (step length after the correction of multiple scattering)
- **Increment** of momentum and energy between **PreStepPoint** and **PostStepPoint**.
(Note: to get the energy deposited in the step, you cannot use this ‘Delta energy’.
You have to use ‘Total energy deposit’ as below.)
- Increment of position and time between **PreStepPoint** and **PostStepPoint**
- **Pointer** to **G4Track**
- **Total energy deposited** during the step - this is the sum of
the energy deposited by the energy loss process, and the energy lost by secondaries
which have **NOT** been generated because each of their energies was
below the cut threshold
- Energy deposited not by ionization during the step
- Secondary tracks created during tracking for the current track.
NOTE: all secondaries are included. NOT only secondaries created in the
CURRENT step.

復習 ステップの両端点で得られる情報

- G4Stepの始点と終点は'G4StepPoint'クラスで表現される。
 - ✓ G4StepPointオブジェクトは始点**PreStepPoint**と終点**PostStepPoint**での粒子の物理状態、位置などの情報を保持する
 - ✓ **下の図で物体Aの中にあるStep1の終点が物質境界面にある場合、終点は境界面の先にある物体Bの中に入ったと解釈する**
- ステップの両端のステップポイント を得るメソッド
 - G4StepPoint* preStepPoint = aStep->GetPreStepPoint(); 始点
 - G4StepPoint* postStepPoint = aStep-> GetPostStepPoint(); 終点



G4StepStatus と 境界判断

- 'G4StepPoint' から現stepが決定された理由を知る事が出来る
- Stepのどちらか一端が物体の境界にある判定にはStepPoint の状態'*fGeomBoundary*'を使う
 - ✓ 物体Aにあるステップ "step1" のPostStepPoint の状態が'*fGeomBoundary*'なら、ステップは物体Aの境界に到達した。
 - ✓ If(step1-> GetPostStepPoint() -> GetStepStatus() == fGeomBoundary) {...}
 - ✓ 物体 B にあるステップ "step2" のPreStepPoint の状態が'*fGeomBoundary*' なら、ステップは物体Bの境界から開始した。
 - ✓ If(aStep-> GetPreStepPoint() -> GetStepStatus() == fGeomBoundary) {...}
- 他にもStepの状態はG4StepStatus列挙型(Enumerator)で以下の列挙定数が定義されている:
 - ✓ **fWorldBoundary**: Stepがworld境界に到達した
 - ✓ **fGeomBoundary**: Stepの端が物質境界面で決められた
 - ✓ **fAtResDoltProc, fAlongStepDoltProc, fPostStepDoltProc**:
Stepがそれぞれの相互作用過程で決定された
 - ✓ **fUserDefinedLimit**: など

[注] G4StepStatus値の使用例
は、例えばGeant4 home pageの
FAQのFAQ.4参照

PreStepPointから得られる情報

G4Step * aStep として このステップが在る物体を「現物体」と呼ぶことにする

- G4StepPoint* point1 = aStep->GetPreStepPoint(); ステップの開始点

今現物体に入ったところか、そのジオメトリは何で、コピー番号は、入射点の座標などを入手するためには必ずこちらを使わねばならない。

if (point1->GetStepStatus() == **fGeomBoundary**) ステップが現物体に入ったところ

つまり、このステップが現物体中での**最初のステップ**である

{ この時、次のように現物体の情報を得る

- G4ThreeVector worldPos1 = point1->**GetPosition()**; 入射点の世界座標
- G4VPhysicalVolume* physVol = point1->**GetPhysicalVolume()**; 現在の物理物体 YAME
- G4int copyNum = physVol->**GetCopyNumber()**; そのコピー番号
- G4String name = physVol->**GetName()**; その名前
- G4LogicalVolume* logVol = physVol->**GetLogicalVolume()**; その論理物体
- G4Material* material = logVol->**GetMaterial()**; その物質
- G4double mass = point1->**GetMass()**; 物体の質量
- G4double kinE = point1->**GetKineticEnergy()**; 入った時の運動エネルギー

などなど

}

PostStepPointから得る情報

- G4StepPoint のメソッドは、メソッド名は同じであっても PreStepPoint とPostStepPointで得られるものが違う。このステップが終了した事由、ジオメトリ境界に達した、ステップを終了させたプロセスは何か、などの情報はこちらを使わねばならない。

```
G4StepPoint* postStepPoint = aStep->GetPostStepPoint();           ステップの終了点
• G4VProcess* aProcess = postStepPoint->GetProcessDefinedStep();   今のステップを決めたプロセス、たとえば、対消滅やBraggピークの端点など。
if (postStepPoint->GetStepStatus() == fGeomBoundary)              ステップが現物体の境界に達した。つまり、現ステップが現物体内での最後のステップである
• const G5TouchableHandle& postStepTouch = postStepPoint->GetTouchableHandle();
• const G4VPhysicalVolume* nextVolume = postStepTouch->GetPhysicalVolume(); 次の物体 (物理物体)
• などなど
```

G4StepPoints.hh を見る 一度はソースに目を通しておくとよい

- (x, y, z, t)
- (px, py, pz, Ek)
- Momentum direction (unit vector)
- Safety
- Beta, gamma
- Polarization
- Step status
- Pointer to the physics process which defined the current step and its DoIt type
- Pointers to physical volumes
- Pointer to the physics process which defined the previous step and its DoIt type
- Total track length
- Global time (time since the current event began)
- Local time (time since the current track began)
- Proper time

G4Track G4Stepとは違う情報

- まず、ステップからトラックを得る

```
const G4Track* track = step->GetTrack();
```

- トラックは物理的オブジェクト（ステップは幾何学的オブジェクト）

- トラックから得る物理量

- **const** G4ThreeVector& momentum = track->GetMomentum();
- **const** G4double kinEnergy = track->GetKineticEnergy();
- **const** G4double globalTime = track->GetGlobalTime();
- など

G4Track は現在のエネルギー、運動量、位置、時間などの物理量と質量、電荷、寿命など静的な粒子の物理状態を持つ。

G4Trackを変更できるのはプロセスだけである。

- (x, y, z)
- Global time (time since the event was created)
- Local time (time since the track was created)
- Proper time (time in its rest frame since the track was created)
- Momentum direction (unit vector)
- Kinetic energy
- Accumulated geometrical track length
- Accumulated true track length
- Pointer to dynamic particle
- Pointer to physical volume

G4Track.hh 続き

- Track ID number
- Track ID number of the parent
- Current step number
- Track status
- (x,y,z) at the start point (vertex position) of the track
- Momentum direction at the start point (vertex position) of the track
- Kinetic energy at the start point (vertex position) of the track
- Pointer to the process which created the current track

トラックーとカロリメータの場合

aStepから様々なジオメトリ情報を得る

- aStepの両端を得る

```
const G4StepPoint* preStepPoint = aStep->GetPreStepPoint();
```

```
const G4StepPoint* postStepPoint = step->GetPostStepPoint();
```

- 両端の世界座標を求める

```
const G4ThreeVector& preStepPosition = preStepPoint->GetPosition();
```

```
const G4ThreeVector& postStepPosition = postStepPoint->GetPosition();
```

- ジオメトリ情報を得るためのタッチャブルハンドル

```
const G4TouchableHandle& preStepTouch = preStepPoint->GetTouchableHandle();
```

- Physical Volume

```
const G4VPhysicalVolume* physVolume = preStepTouch->GetVolume();
```

- その名前

```
const G4String& name = physVolume->GetName();
```

- そのコピー番号

```
const G4int copyNumber = preStepTouch->GetCopyNumber();
```

- その論理ボリューム

```
const G4LogicalVolume* logVolume = physVolume->GetLogicalVolume();
```

- 親の(depth = 1)物理ボリューム

```
const G4VPhysicalVolume* momVolume = preStepTouch->GetVolume(depth=1);
```

Trackerのためのコードの断片

- a. 荷電粒子だけを選ぶ
 - ✓ if (aStep->GetTrack()->GetDefinition()->GetPDGCharge() != 0.) {}
- b. 粒子の名前を選ぶ
 - ✓ If(aStep->GetTrack()->GetDynamicParticle()-> GetParticleDefinition()->GetParticleName() == "proton"){}
- c. 一次粒子のステップを選ぶ
 - ✓ If(aStep->GetTrack()->GetTrackID()==1) {}
- d. 物体に入った最初のステップを判定する
 - ✓ if (preStepPoint->GetStepStatus() == fGeomBoundary){}
- e. 今の物体中での最後のステップかどうかを判定する
 - ✓ if (postStepPoint->GetStepStatus() == fGeomBoundary)
- f. 入射点の世界座標を求める
 - ✓ G4ThreeVector worldPosition = preStepPoint->GetPosition();
- g. ステップが在るジオメトリを知る
 - ✓ G4PhysicalVolume* aStep->GetPhysicalVolume()
- h. 粒子が止まった
 - ✓ If (aStep->GetTrack()->GetTrackStatus() == fStopAndKill) {}

カロリメータのエネルギー損失のコード断片

- 一つの論理物体(名前 'logCylinder') 内でのenergy deposit エネルギーデポジットを求めたいとしよう。各ステップで得られる個々の**エネルギー損失を足し上げて**、イベントの最後にそれらを総計して格納すればよい。
- イベントを開始する
 1. ステップでは、
 - すべてのphysVol=preStepPoint->GetPhysicalVolume() で物理物体をgetし、
If (physVol-> GetLogicalVolume()-> GetName() == 'logCylinder') なら
energyDeposit = aStep -> **GetTotalEnergyDeposit()**
 2. イベントの終わりは、GetStepStatus()でジオメトリ境界に到達したかどうかで判定する

ステップの端点で発生した二次粒子の内
一定の閾値以下のものはそれ以上ステップングを
せず、単にそのエネルギーをここに繰り込む

ユーザ・アクション

ユーザアクションなければスコアリング無し

■ Geant4の既定の実行モードでは一切のスコアリングは行われない

- 3つの必須クラス(Geometry, Physics, Primary Particle)がG4RunManagerに渡されれば粒子輸送(transport/tracking)を実行できる
 - ✓ /run/beamOnでシミュレーションが開始され、指定したイベント数が終わったら、実行の最小情報を書き出した後、黙ってシミュレーションを終了してしまう
 - Verbose モードを上げれば詳しい経過情報を文字方法として得ることはできる、
 - 可視化をすれば絵としての様子はわかるだけだ。

■ Geant4ではスコアを取得し、蓄積し、外部に出力するために、シミュレーションの実行段階に応じてユーザが介入できるように抽象クラスG4UserXxxxAction (Xxxx = Stepping, Event, Run などのユーザ・アクション)が用意されている。スコアリングではユーザはそれらの関数を必要なだけ実装しなければならない。

スコアリングに際してのユーザアクション

- 実際の測定器はさまざまな物質と形状の構造物(ジオメトリ)の検出器の組み合わせであり、それぞれ位置測定、エネルギー測定などのスコアリングを必要とする。
 - 検出器毎に異なるスコアを取らねばならない。
 - 検出器によっては測定器の容器などの物質がある
 - 検出器を支えるためのいわゆるドン殻の存在が位置測定の分解能に影響を与えたり(例えば多重散乱の影響)エネルギー分解能を劣化させたり(例えば、カロリメータ周りの支持物体)する。Geant4のスコアリングでは、どのような要素が測定器の性能に影響を及ぼすかを見積もるのは重要なミッションの一つである。
- イベントごとさらにステップやトラックごとにスコアを入手せねばならないのが普通だ。
 - たとえば、位置測定においては、測定機中で様々な粒子・物質相互作用のため多数の二次3次粒子が発生するが、対象とする一次粒子のみを選びそのステップから直接位置を求めねばならない。
 - 例えば、エネルギー測定でエネルギー分解能を見積もるには、イベントごとの量子的不確定性を考えてエネルギーをスコアせねばならない。

Geant4 が用意するユーザ・アクション

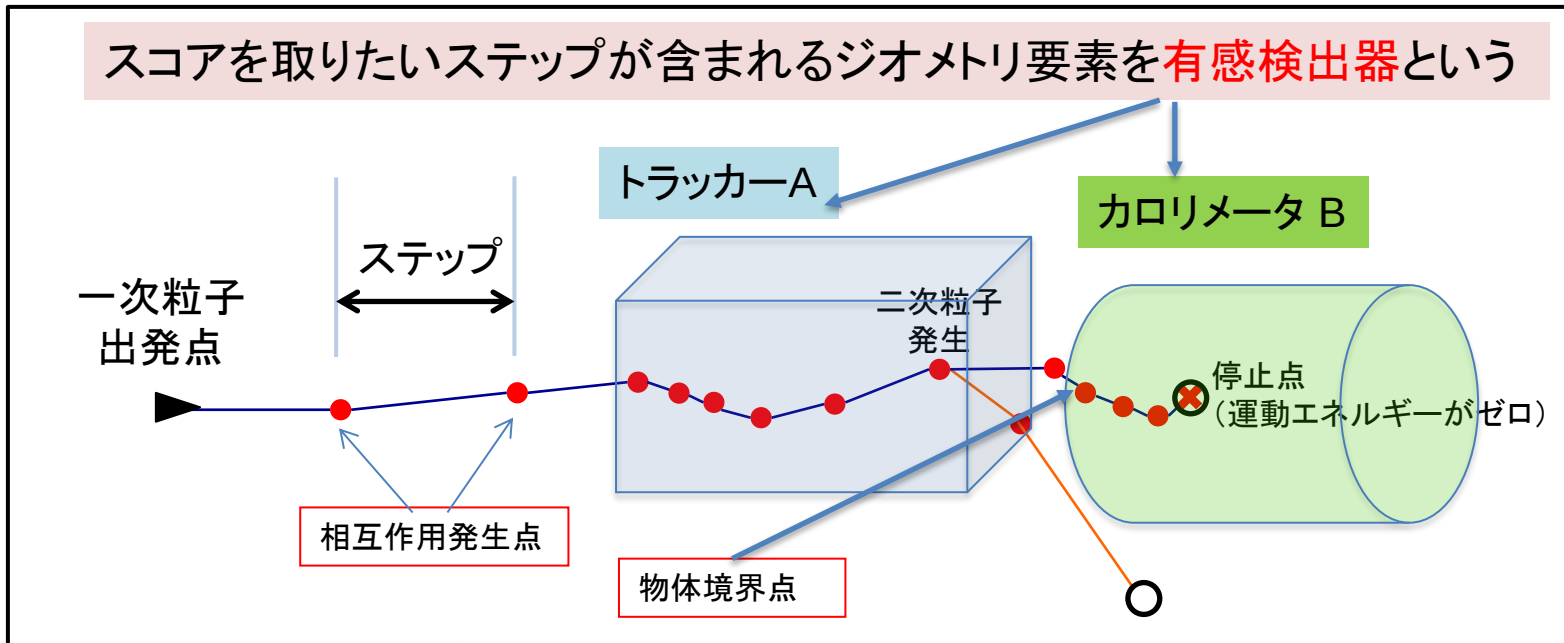
- スコアリングやシミュレーションの流れに介入するために5つのオプションユーザ・アクションが用意されている。このうち1だけがスコアの取得に関わる。

1. G4UserSteppingAction ステップ毎にスコアを得る
 2. G4UserEventAction イベントごとのデータとりまとめ
 - BeginOfEventAction(const G4Event*)
 - EndOfEventAction(const G4Event*)
 3. G4UserRunAction ラン毎の集計など
 - BeginOfRunAction(const G4Run*)
 - EndOfRunAction(const G4Run*)
 4. G4UserStackingAction
 5. G4UserTrackingAction 例えば、トランスポートが必要でない二次粒子を殺すなど
- メインプログラムの中でG4VUserActionInitializationを実装してユーザアクションを初期化せること

SensitiveDetector: 有感検出器

スコアを得たいステップとジオメトリの関係

- シミュレーションは粒子進路に沿ったステップ(step)単位で進行する
 - ✓ ステップ単位で粒子と物質の相互作用(物理: physics)が考慮される
 - ✓ 粒子が構造物の**境界面**を通過すれば、通過点で一つのステップは終了
 - ✓ 粒子の1ステップ長だけの時空間移動 (transportation) も相互作用として扱う
 - ✓ 粒子は運動エネルギーがゼロになれば**停止**し、ステップの進行は終了
- **G4step**オブジェクトはstepの長さ、エネルギー損失などstep全体に関する情報を提供する
 - ✓ ただし、今のステップにある**情報は次のステップでは失われる**



SensitiveDetector (有感検出器)

- すべての情報はステップ(G4Step)から手繰り寄せることができるのだが、
- 複雑な測定機システムは、支持体などドンガラの構造物と、スコアを提供する検出器に分かれる。検出器内部にあるステップだけがスコアリングの対象となる。
 - つまり狙うスコアの取得には2つの課題がある。
 - ① ステップが狙っている**検出器の中にある**と判定すること
 - ② そこで欲しい**ヒット情報を取得**すること
- Geant4はスコアリングを助けるいくつかの方策を提供している。中でもお勧めなのが**Sensitive Detector (有感検出器)**である
 - ステップが有感 sensitive と指定された検出器ジオメトリの中に在ることを**自動的に判定**し、①をユーザが調べる手間は不要とする。
 - ②の課題を実行する**手続関数を自動的に呼ぶ仕組み**を提供する。
- トラックやカロリメータなどそれぞれ別個の有感検出器にできる。ジオメトリ要素を有感に指定する方法も提供される。
- 抽象基底クラス **G4VSensitiveDetector** を継承して実装することになる

SensitiveDetectorの3つの関数

- **G4VSensitiveDetector**を継承して自分のSensitiveDetectorクラスを実装するには、3つの仮想関数を override 実装する。

1. ProcessHits(G4Step* aStep, G4TouchableHistory*) 関数はスコアリングの本体を記述する

- 検出器中でステップの発生毎にこの関数が自動的に呼ばれる。
 - この関数でG4StepやG4Trackからヒット情報を得るコードを書く。
 - 二番目のG4TouchableHistory引数はOBSOLETEなので使わない
- ◆ 取得したスコアの蓄積については場合による。詳しくはその2で説明する。
 - ステップから得た物理量をそのまま保存するだけでよい場合もある。例えば入射点の座標。
 - ハンズオン P10のPixel_Four_Output
 - スコアによってはすべてのステップにまたがって積算すべきものもある。
 - 例えばイベント当たりのエネルギー損失なら一次粒子だけでなくすべての2次3次粒子・・のステップからのdE/dXを加算する必要がある。
 - ハンズオン P09のBGO_Two_Output で電磁シャワーのエネルギーを見たいとき
- ◆ 1イベント毎に多種多様なスコアを取得する場合には、ヒット情報を格納するオブジェクトを用意せねばならない。(後で詳しく説明する)
 - ハンズオンのSimData クラス ではヒットを std::tupleに格納しイベントのヒットを格納するために std::vectorを使う。

SensitiveDetectorの実装 続き

2. `Initialize(G4HCofThisEvent* hce)` 関数

- イベントの最初に呼ばれる
 - 簡単なスコアリングなら目的に合わせて自前のデータ構造を使えばよい。その初期化はここで行う。
 - 必要ならば、このイベントのヒットコレクションHitCollectionOfThisEventを初期化する。ただし、イベントにまたがってヒット情報を集計するのはユーザの仕事となる。本講座では自前のヒットコレクションを使う手法を解説する。

3. `EndOfEvent(G4HCofThisEvent* hce)`関数

- イベントの最後に呼ばれる
 - ーイベントにまたがってステップからのスコアを処理する場合、例えば、エネルギーの積算など、イベント単位でスコアをFillするのに使う。ハンズオンせはSimData の関数 `Fill()`
 - 必要ならばG4HitCollectionofThisEvent(このイベントのヒット集合)を使う。今回は扱わない。

この3つの関数を実装したクラスを ハンズオンP09, P10 ではSensitiveVolume.cc/hh としている

有感指定の対象は論理ボリューム

■ 物理ボリュームと論理ボリューム

● LogicalVolume

- ジオメトリの講義にあったように、論理ボリュームを並進回転移動して物理ボリュームを作ったり、Replicaを作ったりできる。一般に、どのような測定器でも同じユニットを複製して多数配置することが多い。
- したがって、ユニットを構成する**論理ボリュームを有感検出器とする**のが合理的である。
- ユーザのジオメトリ・クラスの `ConstructSDandField()` または `Construct()` の中で論理ボリュームを有感検出器に登録する
- 有感検出器には**ユニークな名前**を付けられ、後で別のクラスからも引用できる。

■ 有感検出器の登録

● G4SDManager

- 階層構造に配置される種々の検出器の登録を管理し、区別するために**シングルトン・マネージャ**である。SensitiveDetector をその**名前**で呼び出すために、`FindSensitiveDetector("name")` を使うことができる。

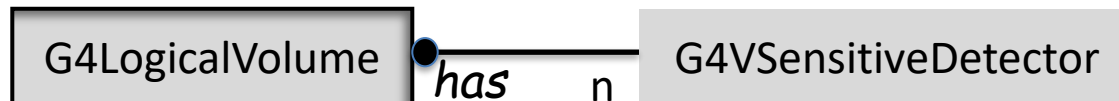
■ ジオメトリの階層構造を探し歩く

● TouchableHandle

- 複製された物理ボリュームを識別するために使う。

ジオメトリで論理物体を有感検出器に指定するには

■ G4LogicalVolumeはG4VSensitiveDetectorを持てる



- 論理物体を有感検出器として指定するには[G4VUserDetectorConstruction](#)を実装したユーザの構造物幾何形状クラスの中で、[ConstructSDandField\(\)](#)関数の中で記述する。用意するものは
 1. 論理物体のインスタンス
 2. 自作の有感検出器のインスタンス
 3. G4SDManager (Sensitive Detector Manager)へのポインタ
- 実際に書かなければならないコードは一つのSensitiveDetectorにつき4行だけである。次に実際のコードで説明する。

論理物体を有感指定する具体的手順

- 論理物体を有感検出器として指定するには[G4VUserDetectorConstruction](#)を実装したユーザの構造物幾何形状クラスの関数[ConstructSDandField\(\)](#)または[Construct\(\)](#)関数の中で記述する。
- SensitiveVolumeクラスのインスタンス”aSensiVol”を作る際に、検出器の階層木構造を取り入れて一意的な名前を与えることができる。階層を仕切るには”/”を使う。たとえば、
 1. [SensitiveVolume* aSensiVol = new SensitiveVolume{ “Pixel” };](#)
 - 有感検出器は名前で識別できる。会話型セッションではコマンドで使える
/hits/inactivate Pixelのように
 2. 有感検出器マネージャG4SDManagerのインスタンスを呼び出し、そのAddNewDetector()関数に aSD を与える。
[auto SDMan = G4SDManager::GetSDMpointer\(\);](#)
[SDMan -> AddNewDetector\(aSensiVol \);](#)
 1. G4VUserDetectorConstruction クラスの SetSensitiveDetector() 関数に論理物体へのポインターを与えると、その論理物体が有感となる
[SetSensitiveDetector\(logVol_PixElmt , aSensiVol, true \);](#)

ハンズオンP09/Bragg_SD/Bragg_DataStoreでは水ファントムを薄切りにしてその1枚を有感指定する。
ハンズオンP10/Pixel_Four_Output ではReplicationした最小のPixel要素logVol_PixElmt 50個を一枚のピクセル検出器とし、それを4個並進回転コピーしてトラッカー検出器としている

多機能検出器(MultiFunctionalDetector) と原始スコアラ(Primitive Scorers)

既成品の有感検出器

原始スコアラ(Primitive scorers)

- Geant4ライブラリにはG4VSensitiveDetectorを継承した多機能検出器 G4MultiFunctionalDetector が提供されている。
- これで間に合うならば、自前の有感検出器クラスを作らなくてよい
- Geant4が用意している既成の具象スコアラ PrimitiveScorer とは、
 - ✓ エネルギー付与や吸収線量のようなよく使われているものでコマンドベーススコアリングのものとほぼ同じである。
 - ✓ ステップ毎のスコアリングは不要で、イベント単位またはラン毎でよい場合
 - ✓ 多機能検出器に一つまたは複数の原始スコアラを割り付けることができる。
- ヒットデータはHitCollectionOf This Event (HCofThisEvent) に格納される。
 - 有感物体のcopy番号やレプリカ番号(キー)とそこでのスコア量の対を要素とする連想リスト(`std::map`) が使われる。
 - キーから値を取り出すイテレータを使えば、容易にデータ集合を操作できるのでイベントの終わりにスコアの積算や集積作業を自分で書かねばならない。
- 抽象基底クラスG4VPrimitiveScorerを具体化すれば自分のスコアラが作れるので、具象原始スコアラのコードを読んで勉強すれば、自前のスコアラを作るのための勉強になる。

PrimitiveScorerの実装例

Basic example B3a の実装例

```
void DetectorConstruction::ConstructSDandField()
{
    G4SDManager::GetSDMpointer()->SetVerboseLevel(1);
    // declare crystal as a MultiFunctionalDetector scorer

    auto cryst = new G4MultiFunctionalDetector("crystal");
    G4SDManager::GetSDMpointer()->AddNewDetector(cryst);
    G4VPrimitiveScorer* primitiv1 = new G4PSEnergyDeposit("edep");
    cryst->RegisterPrimitive(primitiv1);
    SetSensitiveDetector("CrystalLV", cryst);
}
```

SensitiveDetector は自作しなくてよい。“crystal” の論理ボリュームをSDとし、EnergyDeposit スコアを“edep”という名前でスコアしてHit Collection of This Event (HCofThisEvent)に一時保管する。
ここまでは自動的である。

だが、ヒットコレクションから“edep”を取り出し、ランが終わるまで集計してランの終わりに出力するのは自分で方法を考え、実装せねばならない。

用意されている具象原始スコアラ 一覧から抜粋

- 具象原始スコアラ(Guide 4.4.5 Concrete classes of G4VPrimitiveScorerから抜粋)
 - ✓ トラック長 : Weighted()メソッドでステップ毎に重みを付けられる
 - G4PSTrackLength
 - G4PSPassageTrackLength 通過したトラックだけ
 - ✓ エネルギー付与 : 粒子の重みはステップ毎に付ける
 - G4PSEnergyDeposit
 - G4PSDoseDeposit
 - G4PSChargeDeposit
 - ✓ Current: G4BoxのZ平面またはG4Sphereの内側の球面に到達したトラック数
 - G4PSFlatSurfaceCurrent
 - G4PSPassageCurrent
 - G4PSSphereSurfaceCurrent
 - G4PSCylinderSurfaceCurrent
 - ✓ Flux: $W / \cos \phi / A$ 重み、法線への射影、面積
 - G4PSFlatSurfaceFlux
 - G4PSSphereSurfaceFlux
 - G4PSCylinderSurfaceFlux

コマンドベーススコアラー

パラレルワールドとコマンドベーススコアラ

■ パラレルワールド(parallel world)とは

- 今まで学んだMass(物質)ワールドはGeant4の構造物幾何形状で定義されるもので、構造物を構成する立体の形状、物質、寸法、それらの空間配置などの情報を持つ
- 一方、パラレル(parallel)ワールドはマスワールドとは独立に定義出来て(手引き4.7章)
 - マスワールドと同様にパラレルワールドで粒子輸送(navigation)が行われる
 - マスワールドと同様パラレルワールドの境界でもステップは制約される
 - パラレルワールドではスコアするメッシュをマスワールドとは無関係に設定出来る
 - 複数のパラレルワールドが設定出来、それらは重なっても良い

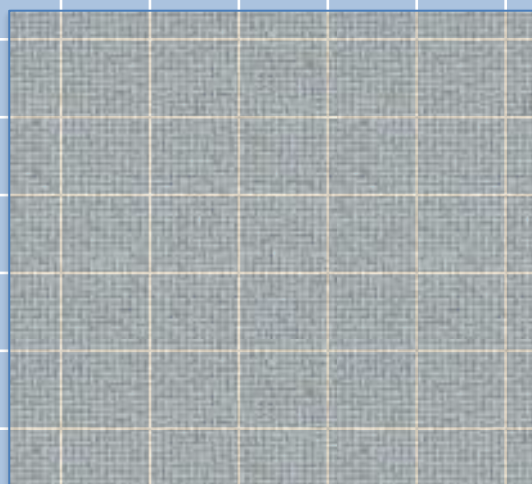
■ コマンドベーススコアラはパラレルワールドを使っている

- パラレルワールドをメッシュで分割し、メッシュごとにスコアリングできる。
- コマンドだけですべてのスコアリング処理ができる
- 先述の具象原始スコアラと同様のスコアラが提供されているが、イベント単位ではなくラン単位でスコアが得られる

マスワールドとパラレルワールド

パラレルのメッシュ

マスの世界



パラレルワールドのメッシュとマスワールドの物体の境界が一致しないメッシュでは正しいスコアが得られないことがある

マクロファイル例 電磁シャワーの発達プロフィールを見る

```
#### Example of cylinder mesh #### /score/ コマンド
## define scoring mesh
/score/create/cylinderMesh cylMesh_1 メッシュに名付け “cylMesh_1”
/score/mesh/cylinderSize 50. 100. mm シリンダー寸法 半径 50mm 長さ 200mm
/score/mesh/translate/xyz 0 0 0. mm シリンダー移動 移動せず
# R Z Phi division
/score/mesh/nBin 1 100 1 メッシュの区切り R方向 1, Z方向 100,  $\phi$ 方向 1
## scoring quantities
/score/quantity/doseDeposit dDep ドーズスコア名 “dDep”
/score/quantity/energyDeposit eDep エネルギー付与スコア名 “eDep”
/score/close スコア閉じる
/score/list
#/control/verbose 2
/tracking/verbose 0
## define gun and start a run
/gps/particle e- 五月蠅すぎるときはレベルを上げる
/gps/energy 10 GeV 10GeV 電子 位置と方向指定
/gps/position 0 0 -20 cm シリンダーの両端のZ -100mm, +100mm なので
/gps/direction 0 0 1
/vis/disable
/run/beamOn 100 ラン開始
#/vis/enable
## Dump scores to a file
/score/dumpQuantityToFile cylMesh_1 eDep eDep.csv CSVファイル“eDep.csv”出力
```

処理の流れ

- ✓ /score/コマンドで
1. メッシュを選び、
 2. その寸法と位置
 3. ビン数を決め
 4. スコアを選ぶ

1. 入射粒子を決めて
2. beamOn

CSVファイルへ出力

シャワーの発達の様子を調べる例 CSV出力 -> Excel

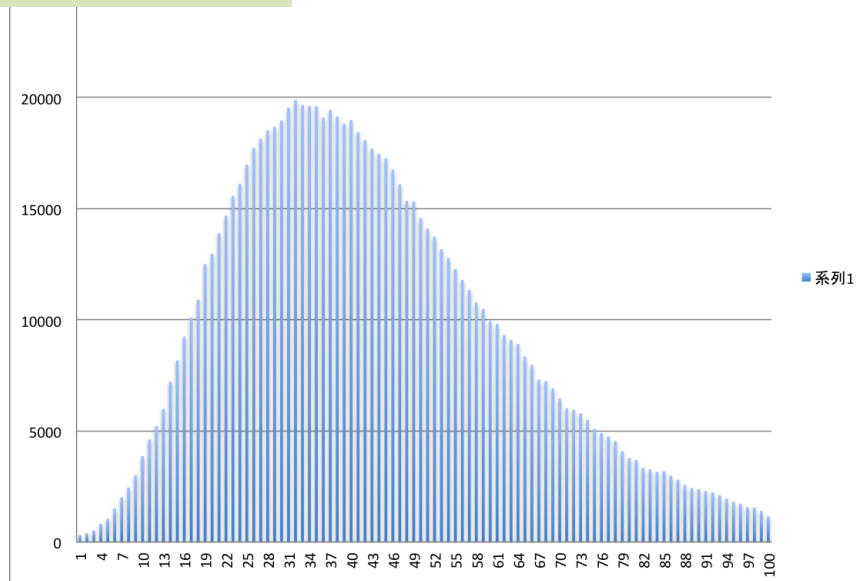
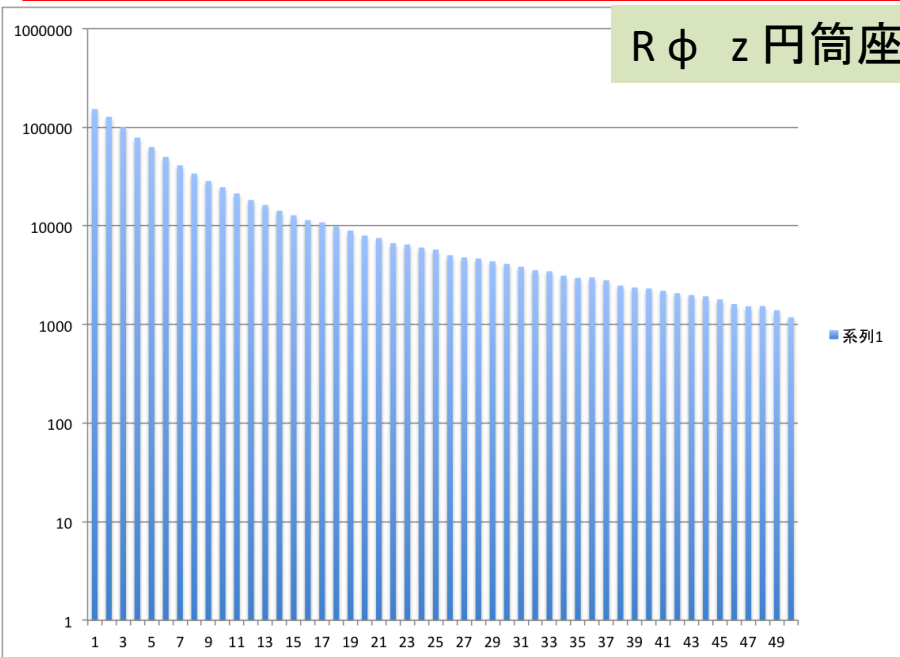
半径 50mm 長さ 200mm BGO単結晶
入射電子 10GeV

横漏れ調べのマクロファイル
R分布 1mmステップ

深さ方向漏れマクロファイル
Z分布 2mmステップ

R ϕ z 円筒座標メッシュの例

Depth- energy Deposit



総デポジットエネルギー 9.369 GeV

講義 その2へ続く